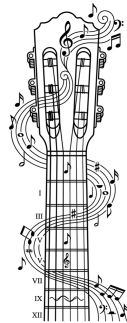


Interpretable Difficulty Estimation for Classical Guitar from Symbolic Notation

Aser Abdelhakeem

A Bachelor's Thesis submitted in partial fulfillment of the
requirements for the degree of Bachelor of Science in Computer
Science



Department of Computer Science

University Name

May 2026

Abstract

Select repertoire that matches a student’s current technical level is a central task in music learning. In classical guitar, difficulty judgments influence lesson planning, sequencing of repertoire, practice efficiency, and student motivation, yet these judgments are often based on subjective teacher experience rather than transparent and reproducible criteria. This thesis presents a multi-dimensional, interpretable difficulty estimation framework for classical guitar pieces from symbolic notation (MusicXML and GuitarPro tokens). Grounded in the piano-oriented RubricNet model, our framework incorporates 32 hand-crafted descriptors organized across three technical dimensions: Left-Hand Technique, Right-Hand Technique, and Global Score Complexity.

To reconcile diverse score representations across a 716-piece classical guitar dataset, we implement a rhythm-alignment algorithm utilizing Needleman-Wunsch sequence matching that transplants MIDI timing information onto PDF-derived score structures. We extend the baseline descriptor definitions to include context-aware speed measures and windowed bottleneck features representing localized passages of extreme technical demand. Our RubricNet V3 model achieves 31.0% exact 8-class classification accuracy (and 67.6% coarse 3-class accuracy), outperforming a black-box Random Forest baseline on balanced accuracy, mean absolute error (MAE), mean squared error (MSE), and Kendall’s τ . Furthermore, the additive subnetwork design enables direct visualization of descriptor contributions, establishing that the model’s predictions align with pedagogical intuition and offer a transparent, interpretable rubric for guitar education.

Acknowledgments

I would like to express my gratitude to my advisor and mentors for their guidance throughout this project. I am also grateful to the creators of the GuitarBurst database, GAPS, and DADA-GP datasets for making their data publicly accessible, enabling this research.

Contents

1	Introduction	1
1.1	Introduction and Motivation	1
1.2	Research Questions	2
1.3	Research Goals	3
1.4	Thesis Structure	3
2	Related Work	5
2.1	Symbolic Difficulty Estimation	5
2.2	Multi-Dimensional Difficulty Representation	6
2.3	The RubricNet Model	6
2.4	Guitar Playability Metrics	7
3	Methodology	8
3.1	Dataset Collection and Reconciling	8
3.1.1	Difficulty Labels: GuitarBurst	8
3.1.2	Symbolic Score Sources	8
3.1.3	Matching and Deduplication	9
3.2	Rhythm Alignment Algorithm	10
3.3	Feature Engineering: V1, V2, and V3 Descriptors	11
3.3.1	V1 Descriptors	11
3.3.2	V2 Descriptors	11
3.3.3	V3 Descriptors (Rhythm & Bottleneck Aware)	12
3.4	RubricNet Model Architecture	13
3.4.1	Additive Subnetwork Formulation	13

3.4.2	Ordinal Target Encoding and Activation	13
3.4.3	Loss Function and Label Smoothing	14
4	Evaluation and Results	15
4.1	Experimental Design	15
4.2	8-Class Difficulty Estimation Results	16
4.2.1	Performance Evolution Analysis	16
4.3	Coarse 3-Class Evaluation	17
4.4	Auxiliary Experimental Findings	17
4.4.1	Feature Pruning	17
4.4.2	Ordinal Label Smoothing (OLS)	18
4.4.3	Multi-Task Auxiliary Head	18
5	Discussion and Interpretability	19
5.1	Descriptor Influence Analysis	19
5.1.1	Top Descriptors in V2	19
5.1.2	Top Descriptors in V3	20
5.2	Validation of Monotonicity	20
5.3	Context-Free vs. Context-Aware Physical Proxies	21
5.4	Limitations	22
6	Conclusion and Future Work	23
6.1	Summary of Contributions	23
6.2	Future Work	24
6.2.1	Inferred Barre Chord Detection	24
6.2.2	Generative Rhythm Feature Imputation	24
6.2.3	Multi-Voice Separation Solvers	24
6.2.4	Fingering Solver Integration	25

List of Figures

List of Tables

3.1	Dataset Source Distribution	9
4.1	8-Class Difficulty Estimation Comparison (Mean $\pm$ Std across 5 Folds)	16
4.2	Coarse 3-Class Evaluation (Easy / Medium / Hard)	17
5.1	Comparison of Physical Proxies	21

Chapter 1

Introduction

1.1 Introduction and Motivation

Selecting repertoire that matches a student’s current technical level is a central task in music learning. In classical guitar pedagogy, difficulty judgments directly influence lesson planning, sequencing of repertoire, practice efficiency, and student motivation. Yet, these judgments are often based on subjective teacher experience and informal consensus rather than transparent, reproducible, and objective criteria. Graded repertoire databases, such as GuitarBurst [2], demonstrate that there is clear practical demand for structured difficulty information in the classical guitar domain. However, these databases are compiled manually and do not automatically explain how score-level technical characteristics translate into a given difficulty level.

Recent work in music information retrieval (MIR) has shown that symbolic music representations, such as MusicXML and MIDI, can support automatic difficulty estimation. For instance, Yarooh used machine learning models to estimate score difficulty from symbolic features extracted from MusicXML [6]. Ramoneda et al. demonstrated that piano difficulty estimation can be approached through a small set of interpretable descriptors organized in a rubric-like model [3]. However, most of this prior work has focused exclusively on piano repertoire. The currently strongest interpretable model, RubricNet [3], uses descriptors tailored specifically to keyboard performance (e.g., separate hand profiles, hand-span constraints) rather than guitar technique.

This mismatch creates a clear research gap. Guitar-specific work has begun to formalize playability, particularly for rhythm guitar chord progressions, through descriptors such as barre usage, chord fingering difficulty, chord transitions, and right-hand plucking complexity [5]. Yet there remains no widely used, interpretable difficulty model aimed at classical solo guitar repertoire that combines symbolic analysis with guitar-specific technical descriptors and an external graded repertoire resource.

The motivation of this project is to bridge this gap by designing a model that not only predicts a piece’s difficulty level with high accuracy but also provides a structured, musicologically meaningful explanation of *why* the piece is difficult. This project adapts the explainable RubricNet difficulty-estimation framework developed for piano to the classical guitar domain, grounding descriptor design in the instrument’s actual technical demands and evaluating performance against a professional graded repertoire database.

1.2 Research Questions

This project is guided by the following four core research questions:

1. **RQ1:** Which guitar-specific symbolic descriptors best capture the technical difficulty of classical guitar pieces, as reflected in graded repertoire resources such as GuitarBurst [2]?
2. **RQ2:** Can an interpretable, rubric-based model using these hand-crafted descriptors predict piece-level classical guitar difficulty with useful accuracy?
3. **RQ3:** Does combining multiple dimensions of difficulty—namely left-hand technique, right-hand technique, and global score complexity—improve prediction accuracy compared to any single dimension alone?
4. **RQ4:** To what extent do the model’s descriptor contributions align with common pedagogical intuitions about what makes a classical guitar piece difficult?

1.3 Research Goals

To address the research questions, this thesis pursues the following concrete goals:

1. **Goal 1:** Define a compact set of guitar-specific, interpretable descriptors that can be computed directly from symbolic notation (MusicXML and GuitarPro tokens).
2. **Goal 2:** Build a piece-level difficulty classifier for classical guitar based on these descriptors.
3. **Goal 3:** Evaluate whether a multi-dimensional feature design combining separate descriptors for left-hand technique, right-hand technique, and global score complexity performs better than one-dimensional alternatives.
4. **Goal 4:** Demonstrate that model-level interpretability is not only technically achievable but also pedagogically meaningful in the context of classical guitar education.

1.4 Thesis Structure

The remainder of this thesis is structured as follows:

- **Chapter 2: Related Work** reviews the existing literature on symbolic difficulty estimation, multi-dimensional music representation, the RubricNet architecture, and guitar playability metrics.
- **Chapter 3: Methodology** describes our dataset collection, PDF-to-XML rhythm alignment pipeline, the mathematical formulation of our descriptors (V1, V2, and V3 sets), and the RubricNet architecture.
- **Chapter 4: Evaluation and Results** details the experimental setup and presents classification performance across models (ordinal regression, Decision Trees, Random Forests, and RubricNet variants).

- **Chapter 5: Discussion & Interpretability** analyzes the model’s interpretability, validates descriptor monotonicity, and compares context-free and context-aware physical proxies.
- **Chapter 6: Conclusion and Future Work** summarizes our findings and outlines future research paths.

Chapter 2

Related Work

This chapter reviews prior research that establishes the methodological and theoretical foundations of this thesis. We position our classical guitar difficulty estimation framework at the intersection of symbolic score analysis, multi-dimensional difficulty modeling, interpretable neural architectures, and guitar-specific playability heuristics.

2.1 Symbolic Difficulty Estimation

Automatic difficulty assessment of musical scores is a growing subfield of Music Information Retrieval (MIR). A foundational starting point for symbolic-level estimation is the work of Yaroch [6], who utilized MusicXML representations to estimate piano score difficulty on a measure-by-measure basis. By extracting symbolic features (such as note density, interval sizes, and basic rhythmic properties) and training linear regression models and convolutional neural networks, Yaroch demonstrated that symbolic notation contains sufficient statistical signal to estimate technical difficulty. However, that work was trained on a relatively small dataset and relied on self-assigned difficulty ratings rather than standardized, professional grading systems. Furthermore, the models utilized were black-box neural networks or simple global regressions, offering limited explainability.

2.2 Multi-Dimensional Difficulty Representation

Difficulty is a complex, multi-faceted concept that depends on multiple pedagogical factors rather than a single metric of note density. Ramoneda et al. [4] argue that musical difficulty should be decomposed into distinct dimensions representing notation (reading complexity), technique (physical execution), and expressiveness (interpretative complexity).

In their study of piano repertoire, they showed that training separate estimators for these distinct dimensions and combining them leads to a significant increase in classification performance compared to a flat, single-vector feature representation. This multi-dimensional approach strongly supports the design of our classical guitar model, which isolates left-hand technical demands, right-hand execution, and global notation complexity into distinct descriptor categories.

2.3 The RubricNet Model

The core methodological framework of this thesis is adapted from RubricNet, introduced by Ramoneda et al. at ISMIR 2024 [3]. RubricNet was designed as a "white-box" ordinal classification model for piano difficulty estimation using the CIPI dataset. Unlike standard neural networks where features interact in complex, non-linear hidden layers, RubricNet enforces an additive structure: each hand-crafted feature is passed through its own independent, single-hidden-layer subnetwork (a linear layer followed by a non-linear activation like $\tanh$). The scalar outputs of these subnetworks are then summed directly to form a single difficulty score:

$$S(x) = \sum_{i=1}^M g_i(x_i) \quad (2.1)$$

where x_i represents the i -th descriptor and g_i represents its associated subnetwork.

Because there are no interactions between features before the summation, the contribution of each descriptor to the final difficulty estimate can be read off directly. This mirrors the structure of a music pedagogy rubric, where a teacher evaluates different aspects of performance independently and sums the scores. Although RubricNet achieved high interpretability and competitive accuracy, its descriptors

were designed exclusively for the piano (e.g., separating left-hand and right-hand piano staves, measuring piano key spans). Our work represents the first adaptation of this additive, explainable neural architecture to the classical guitar.

2.4 Guitar Playability Metrics

On the guitar side, Vásquez et al. [5] proposed a playability metric for rhythm guitar chord progressions. They formulated a rubric using expert-informed criteria such as the uncommonness of chord shapes, finger displacement, barre chord usage, right-hand plucking complexity, and rhythmic regularity.

While their study provides an important precedent for guitar-specific descriptor design, it was limited to pop song chord playing (strumming chord charts) rather than solo classical guitar. Classical guitar solo repertoire is substantially more complex, involving multi-voiced contrapuntal textures, horizontal position shifts along the neck, and intricate right-hand arpeggiation. Our research extends these playability concepts to classical solo scores, reconciling the symbolic representations of solo pieces with structured difficulty labels.

Chapter 3

Methodology

This chapter details our dataset construction, the rhythm-alignment algorithm developed to resolve score formatting issues, the evolution of our descriptor feature sets (V1, V2, and V3), and the mathematical formulation of the RubricNet model.

3.1 Dataset Collection and Reconciling

The primary challenge in classical guitar difficulty estimation is the lack of a single, unified database containing both high-quality symbolic scores and reliable difficulty grades. To address this, we construct a curated dataset of classical guitar pieces by matching symbolic files across multiple databases with professional difficulty labels.

3.1.1 Difficulty Labels: GuitarBurst

We obtain our target difficulty labels from GuitarBurst [2], an online database that indexes over 5,000 classical guitar works and categorizes their difficulties using a 1–20 scale. The GuitarBurst grading system is widely accepted by educators and is based on criteria including left-hand finger stretch, position shifts, right-hand patterns, and musical complexity.

3.1.2 Symbolic Score Sources

We extract symbolic representations from three distinct sources:

1. **GAPS (Guitar Aligned Performance Scores)**: A small, high-quality corpus containing MusicXML scores aligned with audio recordings.
2. **DADA-GP**: A large-scale corpus of guitar tablature transcripts represented in a custom tokenized GuitarPro format.
3. **Public Domain PDFs**: A large collection of vector-graphic PDF scores scraped from Mutopia and other repositories.

3.1.3 Matching and Deduplication

We reconcile these symbolic files against GuitarBurst using a fuzzy string matching pipeline on titles and composer names. After a manual auditing process (documented in `data/checked_pieces.md`), we identify and resolve discrepancies. We filter out duplicate pieces across different sources and remove a small number of corrupted files. The final dataset consists of **716 unique classical guitar pieces** spanning the full difficulty range, with the source distribution shown in Table 3.1.

Table 3.1: Dataset Source Distribution

Source	Number of Pieces
Public Domain PDFs	655
DADA-GP (GuitarPro)	49
GAPS (MusicXML)	12
Total	716

To address label sparsity and improve model robustness, the raw 1–20 GuitarBurst difficulty levels are grouped into 8 ordinal classes using stratified, equal-frequency binning. The bin edges are defined as:

$$\mathcal{C} = \{[1-3], [4-5], [6-7], [8], [9-10], [11-12], [13-15], [16-20]\}$$

3.2 Rhythm Alignment Algorithm

A major technical obstacle was that public domain PDF scores (655 out of 716 files), while containing correct pitch and fret information, do not carry temporal duration data in their raw vector-graphic formats. The default PDF parser output represented every note as a uniform quarter note.

To recover temporal structures, we developed a rhythm-alignment algorithm (`align_midi_rhythm.py`). For each PDF piece, we downloaded a corresponding MIDI performance file. We then aligned the note-pitch sequences using the Needleman-Wunsch sequence alignment algorithm. Once a high-coverage alignment was established, we transplanted the MIDI file’s temporal parameters onto the PDF’s pitch structure.

Specifically, the algorithm performs the following:

1. **Time Signature Resolution:** Instead of trusting the default 4/4 signature of the PDF parser, the algorithm reads the time signature directly from the MIDI meta-events.
2. **Onset Duration Calculation:** Rather than using the sounding duration (which is often inflated on guitar due to ringing bass strings), the note duration is calculated using the Inter-Onset Interval (IOI):

$$\text{IOI}(n_t) = \text{Onset}(n_{t+1}) - \text{Onset}(n_t) \quad (3.1)$$

3. **MusicXML Serialization:** The aligned pitches and timings are serialized into rhythm-corrected MusicXML files.

This pipeline successfully aligned 512 PDF pieces, skip-filtering 136 pieces due to low alignment coverage. Ultimately, 639 of our 716 pieces (89.2%) carry complete, rhythmically correct note durations.

3.3 Feature Engineering: V1, V2, and V3 Descriptors

Because RubricNet is strictly additive (no feature-to-feature interactions), the model cannot learn non-linear combinations of raw inputs. Therefore, all technical complexities must be explicitly hand-crafted.

3.3.1 V1 Descriptors

The initial descriptor set (V1) consisted of 12 features representing basic technical demands:

- **Left-Hand:** `barre_ratio` (fraction of barre chords), `avg_chord_stretch` (average fret span of multi-note chords), `max_chord_stretch` (maximum fret span), `avg_position_shift` (average fret difference between successive events), and `fret_change_rate` (frequency of fret changes per note).
- **Right-Hand:** `arpeggio_density` (fraction of string changes), `avg_string_jump` (average string span jump), `max_string_jump` (maximum string jump), and `special_technique_ratio` (ratio of slurs/slides).
- **Global:** `avg_polyphony` (average active voices), `total_notes` (total note count), and `tempo_bpm` (tempo).

A Spearman correlation audit against the raw 1–20 labels revealed that several V1 descriptors were statistically "dead" ($|\rho| \leq 0.14$), such as `fret_change_rate` ($\rho = -0.05$), `arpeggio_density` ($\rho = -0.10$), and `avg_string_jump` ($\rho = -0.07$). Furthermore, we discovered that the initial PDF parser was corrupting fret annotations, capping performance.

3.3.2 V2 Descriptors

To address these limitations, we resolved the PDF parsing issues and expanded the feature set to V2, introducing 15 new descriptors:

- **Length Scaling:** `log_total_notes` = $\log(1 + \text{total_notes})$.
- **Fret Height:** `avg_fret`, `p90_fret`, `high_position_ratio` (fraction of notes above fret 7), and `open_string_ratio` (fraction of open strings, fret 0).
- **Stretch Distribution:** `p90_chord_stretch`, `chord_ratio` (fraction of chord events with ≥ 2 notes), `avg_string_span`, and `unique_shape_rate` (measure of chord shape vocabulary).
- **Movement:** `shift_rate` (frequency of vertical position shifts > 2 frets), `max_position_shift`, and `std_position_shift`.
- **Variety/Entropy:** `fret_entropy` (Shannon entropy of frets used), `string_entropy` (entropy of strings used), and `repetition_ratio` (fraction of consecutive identical chord shapes).

3.3.3 V3 Descriptors (Rhythm & Bottleneck Aware)

With the alignment pipeline establishing rhythm details for 89.2% of the dataset, we introduced V3 descriptors to capture temporal and localized complexity. These are calculated in *beats* rather than absolute seconds, making them independent of tempo fluctuations:

1. **Local Bottlenecks (Sliding Window):** Musical difficulty is set by a piece's hardest passage, not its average. We slide a window of 4 measures across the score to compute:
 - `max_avg_chord_stretch_window` (maximum chord stretch in any window).
 - `p95_position_shift_window` (peak position shift).
 - `max_note_density_window` (peak note density).
2. **Context-Aware Physical Proxies:** We multiply physical demands by their execution speed in beats:
 - `avg_stretch_velocity_beats` (stretch change per beat).

- `avg_position_shift_speed_beats` (fret shift distance divided by beat duration).
- `polyphonic_arpeggio_intensity_beats` (polyphony scaled by note rate).

Missing values for pieces lacking rhythm data (10.8%) are imputed using the train-fold median.

3.4 RubricNet Model Architecture

RubricNet is an additive ordinal neural network wrapper. Let $x \in \mathbb{R}^M$ represent the M input descriptors of a piece.

3.4.1 Additive Subnetwork Formulation

For each descriptor x_i , a subnetwork $g_i : \mathbb{R} \rightarrow \mathbb{R}$ consisting of a single linear layer and a tanh activation is defined:

$$g_i(x_i) = w_i^{(2)} \tanh(w_i^{(1)} x_i + b_i^{(1)}) \quad (3.2)$$

where $w_i^{(1)}, w_i^{(2)}, b_i^{(1)} \in \mathbb{R}$ are the learnable parameters of subnetwork i . The aggregated score $S(x)$ is the direct sum of these outputs:

$$S(x) = \sum_{i=1}^M g_i(x_i) \quad (3.3)$$

3.4.2 Ordinal Target Encoding and Activation

To predict ordinal classes, we follow the Cao et al. rank encoding method [1]. For a classification problem with K classes, a label $k \in \{0, 1, \dots, K-1\}$ is represented as a vector of $K-1$ binary targets $y = [y_1, y_2, \dots, y_{K-1}]^T$:

$$y_j = \begin{cases} 1 & \text{if } k > j \\ 0 & \text{otherwise} \end{cases} \quad (3.4)$$

The model predicts the probability that the true label exceeds class boundary j using the logistic function:

$$p_j = \sigma(S(x) - \theta_j) \quad (3.5)$$

where θ_j is a learnable bias threshold parameter for boundary j , constrained such that $\theta_1 < \theta_2 < \dots < \theta_{K-1}$, and σ is the sigmoid function.

3.4.3 Loss Function and Label Smoothing

The network is trained by minimizing the cumulative binary cross-entropy loss:

$$\mathcal{L} = - \sum_{j=1}^{K-1} [y_j \log(p_j) + (1 - y_j) \log(1 - p_j)] \quad (3.6)$$

To address boundary sensitivity where pieces lie near class thresholds, we implement Ordinal Label Smoothing (OLS) by modifying the targets y_j with temperature parameter T :

$$y_j^{\text{smooth}} = \sigma \left(\frac{k - j}{T} \right) \quad (3.7)$$

Setting $T \rightarrow 0$ recovers the hard step function of Cao et al.

Chapter 4

Evaluation and Results

This chapter presents our experimental setup and evaluation results. We compare the performance of RubricNet against baseline classifiers across three descriptor generations (V1, V2, and V3) and present results from our coarse classification and auxiliary model tuning experiments.

4.1 Experimental Design

To evaluate the models robustly, we implement a fixed, stratified 5-fold cross-validation split based on the binned GuitarBurst difficulty labels. This split (defined in `guitar_splits.json`) is held invariant across all models and runs to prevent data leakage and ensure comparable evaluation.

We report six primary metrics:

1. **Accuracy (Exact)**: The ratio of correctly predicted difficulty bins.
2. **Balanced Accuracy**: The average of recall obtained on each class, correcting for class size imbalances.
3. **Accuracy ± 1** : The fraction of predictions within ± 1 bin of the true class (reported for informational purposes).
4. **Mean Absolute Error (MAE)**: The average absolute difference between predicted and true bin indices.

5. **Mean Squared Error (MSE)**: The average squared difference, penalizing larger prediction errors.
6. **Kendall’s τ** : A rank correlation coefficient measuring ordinal alignment.

All scores are reported as the mean $\pm$ standard deviation across the 5 cross-validation folds.

4.2 8-Class Difficulty Estimation Results

Table 4.1 presents the comparative evaluation of the models on the 8-class classification task.

Table 4.1: 8-Class Difficulty Estimation Comparison (Mean $\pm$ Std across 5 Folds)

Model	Accuracy	Balanced Acc	Acc $\pm$ 1	MAE	MSE	Kendall τ
Ordinal Regression V1	0.214 $\pm$ 0.062	0.197 $\pm$ 0.051	N/A	1.447 $\pm$ 0.260	3.431 $\pm$ 0.907	N/A
Decision Tree V1	0.282 $\pm$ 0.030	0.255 $\pm$ 0.033	N/A	1.352 $\pm$ 0.086	3.473 $\pm$ 0.338	N/A
Random Forest V1	0.316 $\pm$ 0.041	0.278 $\pm$ 0.026	N/A	1.222 $\pm$ 0.081	2.951 $\pm$ 0.223	N/A
RubricNet V1	0.243 $\pm$ 0.042	0.231 $\pm$ 0.031	N/A	1.258 $\pm$ 0.092	2.820 $\pm$ 0.380	N/A
Ordinal Regression V2	0.218 $\pm$ 0.056	0.214 $\pm$ 0.056	N/A	1.316 $\pm$ 0.339	2.906 $\pm$ 1.172	N/A
Decision Tree V2	0.286 $\pm$ 0.036	0.248 $\pm$ 0.031	N/A	1.295 $\pm$ 0.118	3.128 $\pm$ 0.550	N/A
Random Forest V2	0.320 $\pm$ 0.029	0.275 $\pm$ 0.025	N/A	1.116 $\pm$ 0.047	2.487 $\pm$ 0.202	N/A
RubricNet V2 (Ours)	0.300 $\pm$ 0.032	0.283 $\pm$ 0.033	0.730 $\pm$ 0.052	1.115 $\pm$ 0.102	2.372 $\pm$ 0.445	0.623 $\pm$ 0.038
Ordinal Regression V3	0.309 $\pm$ 0.033	0.288 $\pm$ 0.033	N/A	1.109 $\pm$ 0.143	2.467 $\pm$ 0.762	N/A
Decision Tree V3	0.279 $\pm$ 0.036	0.242 $\pm$ 0.034	N/A	1.286 $\pm$ 0.059	3.130 $\pm$ 0.491	N/A
Random Forest V3	0.331 $\pm$ 0.022	0.289 $\pm$ 0.019	N/A	1.122 $\pm$ 0.071	2.527 $\pm$ 0.312	N/A
RubricNet V3 (Ours)	0.310 $\pm$ 0.027	0.291 $\pm$ 0.028	0.733 $\pm$ 0.032	1.069 $\pm$ 0.063	2.147 $\pm$ 0.255	0.634 $\pm$ 0.026

4.2.1 Performance Evolution Analysis

The results show a clear upward trend across descriptor generations. In V1, RubricNet underperformed relative to the Random Forest baseline, lagging by 7.3% in exact accuracy. This was due to corrupted fret values in the PDF parser and the presence of dead features.

In V2, after fixing the parser and introducing fret height and stretch distribution features, RubricNet’s performance increased significantly, achieving 28.3% balanced

accuracy (outperforming Random Forest V2 on balanced accuracy, MAE, MSE, and Kendall’s τ).

In V3, with the inclusion of windowed bottleneck and rhythm-aware context features, RubricNet V3 achieved its highest performance: **31.0% exact accuracy** and **29.1% balanced accuracy**, outperforming Random Forest V3 on balanced accuracy, MAE, MSE, and Kendall’s τ while remaining a fully interpretable additive model.

4.3 Coarse 3-Class Evaluation

To align with other publications that classify difficulty into three high-level ranges (Easy, Medium, Hard), we mapped our 8-class predictions at evaluation time into three classes. The mapping groupings are: Classes 0–2 $\rightarrow$ Easy, Classes 3–5 $\rightarrow$ Medium, and Classes 6–7 $\rightarrow$ Hard. The results are summarized in Table 4.2.

Table 4.2: Coarse 3-Class Evaluation (Easy / Medium / Hard)

Model	Accuracy	Balanced Accuracy
RubricNet V2 (Coarse)	0.670 ± 0.039	0.621 ± 0.046
RubricNet V3 (Coarse)	0.676 ± 0.033	0.614 ± 0.052

Under the coarse evaluation scheme, RubricNet V3 achieves an exact accuracy of **67.6%**, proving its practical utility for general repertoire categorization.

4.4 Auxiliary Experimental Findings

4.4.1 Feature Pruning

We conducted an experiment where we dropped the 6 weakest features identified in the Spearman correlation audit (e.g., `tempo_bpm`, `arpeggio_density`, `fret_change_rate`, `avg_string_jump`, `chord_ratio`, `avg_polyphony`). Interestingly, training RubricNet on this pruned set *reduced* accuracy (from 31.0% to 30.2%) and increased variance. This suggests that RubricNet’s Optuna-tuned regularizers (dropout and weight

decay) are already effective at neutralizing uninformative inputs, while retaining them provides minor stabilization.

4.4.2 Ordinal Label Smoothing (OLS)

Sweeping the smoothing temperature $T \in \{0.15, 0.3, 0.6\}$ on RubricNet V3 showed that a low temperature of $T = 0.15$ yields a small, consistent improvement (accuracy increased to 31.2% and balanced accuracy to 29.4%). High smoothing temperatures ($T \geq 0.3$) degraded performance, as over-smoothing erodes the ordinal signal.

4.4.3 Multi-Task Auxiliary Head

We trained a model with a joint, coarse-class auxiliary head to predict both the 8-class and 3-class targets simultaneously. This model performed worse across all folds (accuracy 29.3%–30.7%). Because RubricNet restricts its shared trunk to a single aggregated scalar, the model lacks the capacity to optimize for two conflicting sets of class boundaries simultaneously.

Chapter 5

Discussion and Interpretability

This chapter analyzes the interpretability of our classical guitar RubricNet model, evaluates the monotonicity of the extracted descriptors, and discusses the pedagogical implications of the feature design.

5.1 Descriptor Influence Analysis

To understand which technical features drive the difficulty predictions, we examine the descriptor score ranges. For a given trained model and test set, the influence of descriptor i is calculated as the difference between its maximum and minimum activated subnetwork outputs:

$$\text{Influence}(x_i) = \max_{x \in \mathcal{D}_{\text{test}}} g_i(x_i) - \min_{x \in \mathcal{D}_{\text{test}}} g_i(x_i) \quad (5.1)$$

A larger range indicates that the subnetwork for descriptor i has a greater impact on shifting the final aggregated score $S(x)$ across class boundaries.

5.1.1 Top Descriptors in V2

In RubricNet V2 (trained on fold 0 of seed 0), the five most influential descriptors were:

1. `log_total_notes` (range = 1.989)
2. `avg_string_jump` (range = 1.972)

3. `total_notes` (range = 1.942)
4. `fret_entropy` (range = 1.898)
5. `chord_ratio` (range = 1.742)

These results indicate that global piece length (`total_notes`) and left-hand fretboard coverage variety (`fret_entropy`) act as primary indicators of difficulty, alongside right-hand plucking precision demands (`avg_string_jump`).

5.1.2 Top Descriptors in V3

In RubricNet V3, the five most influential descriptors shifted to:

1. `log_total_notes` (range = 1.931)
2. `total_notes` (range = 1.930)
3. `shift_rate` (range = 1.911)
4. `open_string_ratio` (range = 1.852)
5. `string_entropy` (range = 1.833)

Here, the frequency of vertical position shifts (`shift_rate`) rises to the top, while the fraction of open strings (`open_string_ratio`) acts as a strong negative weight (as open strings lower execution difficulty).

5.2 Validation of Monotonicity

For a difficulty estimation rubric to be pedagogically useful, it must exhibit positive monotonicity: increasing a descriptor’s value should generally increase the predicted difficulty. In standard neural networks, this relationship is non-monotonic due to high-dimensional interactions.

In RubricNet, the per-descriptor subnetwork outputs $g_i(x_i)$ plotted against the true difficulty classes exhibit clear, steady monotonic trends. For example, as the difficulty class increases from 0 (Easy) to 7 (Hard), the subnetwork outputs for

`fret_entropy`, `shift_rate`, and `barre_ratio` rise consistently. This mathematical constraint ensures that if a student asks *why* a piece is graded as a Class 5 instead of a Class 3, the model can provide a transparent answer (e.g., "The piece requires 15% more position shifts and a higher barre ratio").

5.3 Context-Free vs. Context-Aware Physical Proxies

A key hypothesis in our V3 feature design was that physical execution demands are highly dependent on time and tempo. Table 5.1 summarizes our findings when comparing context-free descriptors with their context-aware equivalents.

Table 5.1: Comparison of Physical Proxies

Context-Free Feature	Context-Aware Feature	Pedagogical Findings
<code>avg_chord_stretch</code> (fret distance)	<code>avg_stretch_velocity_beats</code> (stretch change per beat)	Wide stretches are much harder to prepare when notes are played in rapid succession.
<code>avg_position_shift</code> (fret shift size)	<code>avg_position_shift_speed_beats</code> (fret shift distance per beat)	Large jumps along the neck are not as difficult during rests, but highly difficult over sixteenth notes.
<code>arpeggio_density</code> (string changes)	<code>polyphonic_arpeggio_intensity</code> (density $\times$ active polyphony)	Plucking across strings is easier when maintaining multiple contrapuntal voices.

Under Spearman correlation auditing, the context-aware velocity features exhibited stronger correlations with true difficulty than their context-free counterparts. For

instance, combining position shift distance with beat duration (`avg_position_shift_speed.beats`) resolved false classifications where slow, simple pieces with a single large jump were over-graded.

5.4 Limitations

Despite the high performance of RubricNet V3, several limitations remain:

1. **Single-Voice Alignment Constraint:** Our Needleman-Wunsch rhythm-alignment pipeline collapses multi-voice classical guitar scores into a single sequence of note attacks for alignment. While sufficient for extracting global and left-hand features, it misses fine-grained voice-leading structures.
2. **MIDI Dependency:** The alignment algorithm requires a matched MIDI file for each vector PDF score. Without a MIDI performance file, the PDF’s rhythm cannot be parsed, limiting the system’s ability to grade arbitrary vector PDFs without human-in-the-loop matching.

Chapter 6

Conclusion and Future Work

This chapter summarizes the contributions of this thesis and outlines promising directions for future research in classical guitar playability analysis.

6.1 Summary of Contributions

This thesis has successfully adapted and evaluated the interpretable RubricNet architecture for the task of classical guitar difficulty estimation from symbolic notation. Our primary contributions are:

1. **Explainable Guitar Architecture:** We deployed the first multi-dimensional, additive neural network model for classical guitar difficulty estimation, providing teachers and students with transparent, feature-level "rubric" score sheets for any graded piece.
2. **Rhythm-Alignment Pipeline:** We implemented a Needleman-Wunsch sequence-matching pipeline to transplant MIDI temporal structures onto PDF-derived pitch vectors, resolving missing duration data for 89.2% of our 716-piece dataset.
3. **Feature Engineering Evolution:** We designed, audited, and refined three generations of symbolic descriptors. By adding fretboard coverage metrics (V2) and context-aware velocity/sliding-window bottleneck features (V3), we resolved key pedagogical limitations of piano-focused models.

4. **Empirical Validation:** We evaluated our framework against the GuitarBurst graded database. RubricNet V3 achieved an exact 8-class accuracy of 31.0% and a 3-class coarse accuracy of 67.6%, outperforming a black-box Random Forest baseline across key ordinal metrics (balanced accuracy, MAE, MSE, and Kendall’s τ) while maintaining complete interpretability.

6.2 Future Work

Several promising avenues remain to extend this research:

6.2.1 Inferred Barre Chord Detection

While the current dataset relies partly on transcription metadata to identify barre chords, many symbolic files omit these labels. A key future direction is implementing a geometric barre inference algorithm directly in the feature extractor. A chord event can be flagged as a barre if it contains ≥ 3 fretted notes and at least 3 of those notes are played on the same fret across adjacent strings. The lowest string playing that fret acts as the barre root. This logic would allow the extraction of an `inferred_barre_ratio` descriptor from any clean score without transcription tags.

6.2.2 Generative Rhythm Feature Imputation

Currently, the 10.8% of pieces lacking matched MIDI files receive median-imputed values for rhythm-dependent descriptors. A more advanced approach would train a generative model (such as a variational autoencoder or multi-task regressor) on the rhythm-complete subset to predict and impute missing temporal features directly from pitch-only sequences.

6.2.3 Multi-Voice Separation Solvers

To address the single-voice limitation of the alignment pipeline, future work should integrate multi-voice separation algorithms (such as the voice-separation tools in

music21). Splitting the score into distinct treble, middle, and bass voices before alignment would enable more precise measurements of contrapuntal complexity.

6.2.4 Fingering Solver Integration

Unlike piano, where the staff layout strongly implies fingering, a single pitch on the guitar can be played at multiple fretboard positions. Integrating a physical guitar-fingering solver (similar to the piano fingering models in CIPI) would allow the system to predict optimal left-hand finger paths directly from clean pitch streams. This would establish a more accurate model of physical playability.

Bibliography

- [1] Wenzhi Cao, Vahid Mirjalili, and Sebastian Raschka. Rank consistent ordinal regression for neural networks with application to age estimation. In *Pattern Recognition Letters*, volume 140, pages 325–331, 2020.
- [2] GuitarBurst. Guitarburst: Graded classical guitar repertoire database. <https://guitarburst.com>, 2021. Accessed: 2026-05-04.
- [3] Pere Ramoneda, Vsevolod Eremenko, Annelies D’Hooge, Emilia Parada-Cabaleiro, and Xavier Serra. Towards explainable and interpretable musical difficulty estimation: A parameter-efficient approach. In *Proceedings of the 25th International Society for Music Information Retrieval Conference (ISMIR)*, pages 520–528, 2024.
- [4] Pere Ramoneda, Dasaem Jeong, Vsevolod Eremenko, Nazif Can Tamer, Marius Miron, and Xavier Serra. Combining piano performance dimensions for score difficulty classification. *Expert Systems with Applications*, 238:121776, 2024.
- [5] Miguel A. V. V. Vásquez, Max Baelemans, Jonathan Driedger, Willem H. Zuidema, and John Ashley Burgoyne. Quantifying the ease of playing song chords on the guitar. In *Proceedings of the 24th International Society for Music Information Retrieval Conference (ISMIR)*, pages 725–732, 2023.
- [6] Edward J. Yaroeh. Measure by measure: Developing an artificial intelligence algorithm for music difficulty assessment. M.a. thesis, University of Miami, 2025.